Taller de Proyecto II

Informe de Avance - 11/10/2026

Universidad Nacional de La Plata





EcoSort: Clasificador de Residuos Inteligente

Autores

Juan David Alvarez - 03100/2

Micaela Taini - 03345/2

Francisco Estrada - 02999/3

Taller de Proyecto II (I1118) - Facultad de Informática

2° Cuatrimestre de 2026 - Docente Tutor: Gastón Maron

Índice

1. Descripción General del Proyecto....3
1.1 Modificaciones a partir de evaluaciones de la cátedra....3
1.2 Decisiones adoptadas durante el desarrollo....3
1.3 Alcance de este documento....4
2. Materiales....4
3. Esquema Gráfico del Proyecto....5
4. Grado de Avance....6
4.1 Grado de Avance por Parte....6
4.2 Cumplimiento del Avance de Octubre....7
4.3 Hardware probado, requerimientos de alimentación y planificación....8
4.4 Documentación del Código....9
4.4.1 Código que controla el hardware....9
4.4.2 Código que controla las comunicaciones....10
4.4.3 Código del Sistema y la interfaz web....11
4.4.4 Software para PC o dispositivo móvil....12
4.5 Modelo de clasificación y optimización para la Raspberry Pi....12
4.6 Pruebas realizadas....13
4.7 Problemas encontrados y soluciones....14
4.8 Planificación de las partes faltantes....15
5. Documentación Relacionada....16
5.1 Fotografías del hardware....16
5.2 Capturas de la interfaz web....16
5.3 Video de las pruebas....17
5.4 Bitácora y repositorio....17
6. Bibliografía....17

1. Descripción General del Proyecto

EcoSort es un punto de reciclaje inteligente que identifica, mediante visión artificial, la categoría de un residuo en el momento en que se deposita y lo deriva al contenedor correspondiente sin intervención humana en la clasificación. El Plan de Proyecto entregado el 13/09/2026 estableció como objetivo general clasificar el residuo en una de cuatro categorías (plástico, papel, vidrio y orgánico) y accionar una compuerta física, con la inferencia ejecutándose íntegramente en una Raspberry Pi 3, sin conexión a internet.

1.1 Modificaciones a partir de evaluaciones de la cátedra

Las evaluaciones del docente tutor y de la cátedra modificaron o precisaron el Plan en los siguientes puntos:

- Plataforma física: se definió y cerró con el docente una plataforma fija de cuatro compuertas, una por clase, todas cerradas en reposo. La cátedra confirmó la disponibilidad de cuatro servomotores SG90, lo que reemplazó las alternativas de torreta rotativa y de rampa con aletas.
- Interfaz de monitoreo: a pedido del docente, se reemplazó el uso de una herramienta de monitoreo preexistente (Grafana) por un dashboard propio, desarrollado por el grupo.
- Documentación de la arquitectura: por recomendación del docente, el diagrama también se mantiene como texto versionado en el repositorio.

1.2 Decisiones adoptadas durante el desarrollo

Durante el desarrollo del proyecto tomamos decisiones que precisaron el Plan. En el módulo de la Raspberry Pi fueron:

- Modelo: en lugar de un detector de objetos como You Only Look Once (YOLO), se adoptó un clasificador de imágenes basado en MobileNetV2 con aprendizaje por transferencia (transfer learning), entrenado en Google Colab y exportado a TensorFlow Lite con cuantización a enteros de 8 bits (int8).
- Conjunto de datos: se utilizó TrashNet [4] como línea base. Este conjunto no contiene la clase orgánico, por lo que la construcción de un conjunto propio queda como tarea principal de las próximas semanas.

- Transporte de Datos: se adoptó el protocolo Message Queuing Telemetry Transport (MQTT) con el broker Mosquitto [5] ejecutándose en la propia Raspberry Pi, de acuerdo con el Plan.
- Conteo por objeto: se incorporó un mecanismo de detección de presencia por diferencia de fondo, de modo que cada residuo genere un único evento.

1.3 Alcance de este documento

El presente informe documenta el trabajo realizado en el módulo de la Raspberry Pi: preparación del sistema, captura de imágenes, entrenamiento y optimización del modelo, inferencia, conteo, publicación de eventos por MQTT, un panel de monitoreo de prueba y el diseño del circuito de alimentación. El adaptador hacia la base de datos, el esquema de datos definitivo, el firmware de actuadores y la integración continua están a cargo de otros integrantes y se documentan por separado.

2. Materiales

La Tabla 1 lista los elementos de hardware del proyecto con su estado.

Ítem	**Objetivo**	**Cant.**	**Estado**	**Costo aprox.**
Raspberry Pi 3	Procesador, inferencia y servidor local	1	Disponible	Sin costo
Fuente Cargador con salida microUSB (5 V / 2,5 A,)	Alimentación de la Raspberry Pi y la cámara	1	Disponible	-
MicroSD 64 GB clase A1/A2	Almacenamiento y sistema de archivos	1	Disponible	Sin costo
Cámara USB con lente M12 de enfoque manual	Captura de imagen	1	Provista por la cátedra	Sin costo
Servo SG90	Accionamiento de la compuerta de cada clase	4	Provistos por la cátedra	Sin costo
Fuente de computadora (riel de +5 V)	Alimentación de los servos, separada de la Raspberry Pi	1	Provista por la cátedra	Sin costo

Plataforma de modelado (cartón corrugado o impresión 3D)	Mecanismo de clasificación	1	Pendiente	A relevar

Tabla 1. Materiales del proyecto, estado y costo aproximado

3. Esquema Gráfico del Proyecto

La Figura 1 presenta el esquema del sistema y la ubicación del módulo documentado en este informe. El bloque verde corresponde a la Raspberry Pi y el bloque gris punteado, a un dispositivo externo que recibe los eventos. Los trazos naranja punteados señalan código aún no probado con el hardware.

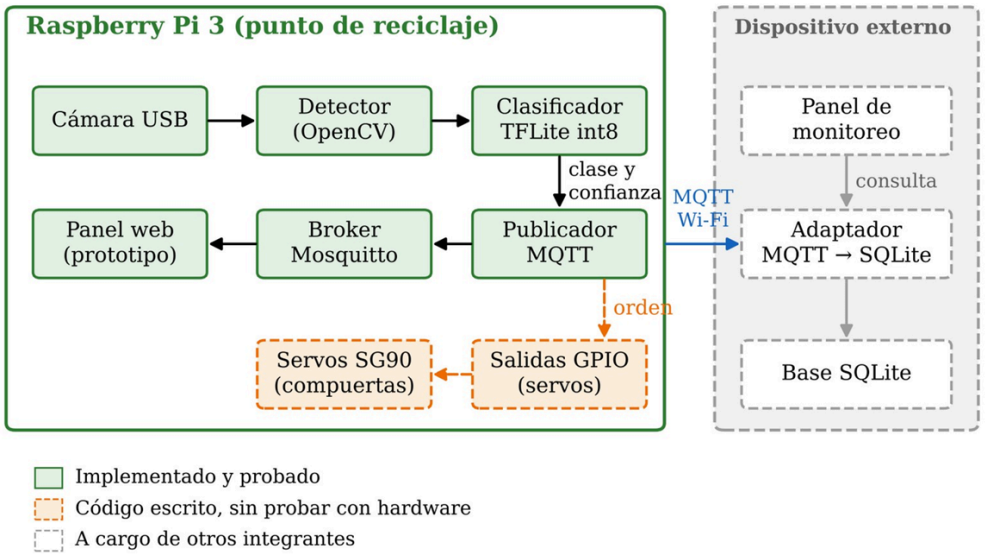


Figura 1. Esquema del sistema EcoSort: módulos en la Raspberry Pi y disponible externo

La cámara USB entrega imágenes al detector, basado en OpenCV, que decide si hay un objeto en la escena. Cuando lo hay, el clasificador TensorFlow Lite (int8) estima la clase y la confianza. El publicador MQTT envía al broker Mosquitto, que se ejecuta en la propia Raspberry Pi, un mensaje por cada residuo contado y otro, más frecuente, con el estado en vivo. El panel web de prueba se suscribe a ese broker, guarda los eventos y los muestra, y un adaptador en un dispositivo externo los almacena en una base SQLite. El módulo de salidas de propósito general (GPIO) traduce la clase detectada en una señal de control hacia el servo de la compuerta correspondiente.

El adaptador y el panel son dos procesos independientes que no se conocen entre sí (ADR 0003, ADR 0007): comparten únicamente el broker MQTT y el archivo eventos.db, en modo WAL de SQLite, que admite un escritor (el adaptador) y lectores concurrentes (el panel) sin coordinación adicional. Si el panel se cae, el adaptador sigue guardando eventos; un reinicio del panel no interrumpe la persistencia.

La Figura 2 muestra las conexiones físicas. Se emplean dos ramas de alimentación independientes para impedir que los picos de corriente de los servos reinicien la Raspberry Pi: la Fuente 1 alimenta la placa y, a través de ella, la cámara USB; la Fuente 2 alimenta únicamente los servos. Ambas ramas comparten tierra (ground, GND), condición necesaria para que la señal de control de modulación por ancho de pulso (PWM) tenga una referencia común. Las señales de control salen de los pines GPIO 13, 18, 22 y 23, que corresponden a los pines físicos 33, 12, 15 y 16.

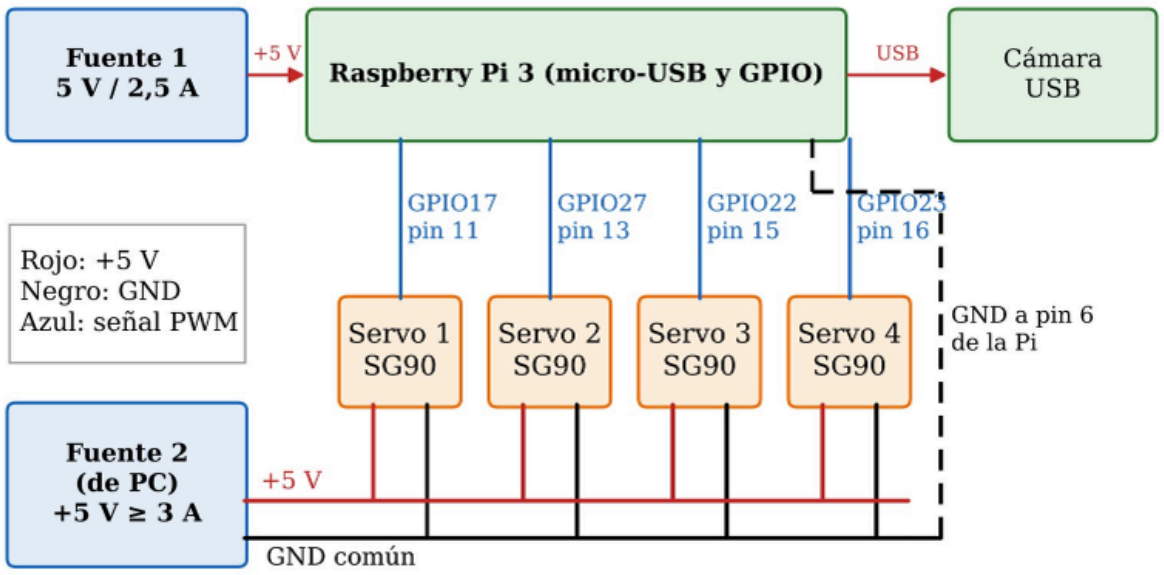


Figura 2. Esquema de conexiones y circuito de alimentación de la Raspberry Pi, la cámara y los cuatro servos

4. Grado de Avance

4.1 Grado de Avance por Parte

La Tabla 2 resume el grado de avance aproximado de cada parte del módulo de la Raspberry Pi. Los porcentajes son estimaciones del grupo respecto de lo previsto para la entrega final y no resultan de una medición.

Parte	**Avance**	**Estado**
Preparación de la Raspberry Pi (sistema operativo, entorno de Python, Mosquitto)	90 %	Operativa y documentada en una guía de instalación; falta automatizar el arranque.
Despliegue de la Pi (Ansible)	60 %	Playbook completo y probado con tests de sintaxis y de plantillas; sin correr contra la Pi real.
Captura de imágenes y ajuste de la cámara	90 %	Captura estable a 640 × 480; vista en vivo con indicador de nitidez para el enfoque manual.
Modelo de clasificación (entrenamiento y exportación)	60 %	Modelo base entrenado con TrashNet; falta el conjunto propio, la clase orgánico y la validación en condiciones reales.
Inferencia en la Raspberry Pi	85 %	Latencia de 31 ms por imagen con el modelo int8; falta calibrar los umbrales con la cámara real.
Detección de presencia y conteo por objeto	75 %	Funcional con fondo estático; sin validación sistemática de eventos falsos.
Publicación MQTT y *broker* Mosquitto	90 %	Eventos, estado en vivo y estado de conexión operativos; con reconexión automática.
Contrato de eventos (schema, validación)	90 %	Versionado y en uso; falta persistir los descartes como telemetría.
Backend del panel (API, SSE)	85 %	API de resumen y de estadísticas operativas; sin autenticación (aceptable en la LAN cerrada).
Despliegue del host (Docker Compose)	90 %	Dos imágenes, con configuración por .env; probado de punta a punta.
Panel web de prueba (conteo, estado en vivo, descarga de datos, reinicio)	80 %	Probado de punta a punta en la Raspberry Pi.
Control de servos (módulo GPIO)	30 %	Código escrito; sin pruebas con los servos conectados.

Circuito de alimentación	70 %	Diseñado y dimensionado en este informe; sin medición de consumo ni montaje.

Tabla 2. Grado de avance aproximado de las partes del módulo de la Raspberry Pi

4.2 Cumplimiento del Avance de Octubre

El Plan de Proyecto estableció los resultados esperados para el Informe de Avance de octubre. La Tabla 3 los contrasta con lo realizado en el módulo de la Raspberry Pi.

Resultado planificado	**Estado**	**Observaciones**
Modelo base exportado a TensorFlow Lite, ejecutándose en la Raspberry Pi y publicando eventos por MQTT	Cumplido	Modelo MobileNetV2 int8 con 31 ms de latencia media; eventos publicados al *broker* local.
Cuatro LED que respondan a un evento de clasificación	No evaluado	Corresponde al área de firmware.
Comunicación MQTT enviando información al tópico correspondiente	Cumplido	Tópicos de eventos, estado en vivo y estado de conexión (sección 4.4).
Esquema de datos de eventos acordado y congelado	Cumplido	Contrato v1 (ADR 0008) con versión explícita (schema_version), vocabulario en schema/clases.json y schema/evento.schema.json; cambiarlo requiere el review de los tres integrantes.
Adaptador MQTT a SQLite y dashboard que muestre eventos en una tabla y un gráfico simple	Cumplido	Adaptador y panel como procesos independientes (ADR 0003/0007); panel con tabla, conteo por clase, una API de agregaciones para graficar.
Circuito de alimentación diseñado y documentado	Cumplido	Diseño en las secciones 3 y 4.3, con tabla de consumo y dimensionamiento.
Repositorio con estructura documentada, integración continua y tablero de tareas	Parcial	Estructura documentada e integración continua; el tablero de tareas no se implementó en esta entrega.

Tabla 3. Cumplimiento de lo planificado para el Avance de Octubre

4.3 Hardware probado, requerimientos de alimentación y planificación

Se probaron con éxito la Raspberry Pi 3, la tarjeta microSD con Raspberry Pi OS de 64 bits, la cámara USB y la red local mediante Wi-Fi. Durante las pruebas de inferencia, la temperatura de la placa fue de 43,5 °C y el indicador de limitación (`vcgencmd get_throttled`) devolvió `0x0`, es decir, sin subtensión ni estrangulamiento térmico.

La Tabla 4 presenta la tensión y la corriente de cada componente, el consumo en el peor caso y el dimensionamiento de las fuentes. Los valores de la Raspberry Pi provienen de la documentación del fabricante [1] y la tensión de operación del servo, de su hoja de especificaciones [2]. Las corrientes de reposo y de bloqueo del servo, y la corriente máxima de la cámara, son valores de diseño asumidos de forma conservadora y deben confirmarse midiendo el equipo real.

Componente	**Tensión**	**Corriente típica**	**Corriente máxima de diseño**	**Fuente**
Raspberry Pi 3 (placa)	5 V	0,40 A [1]	—	Fuente 1
Cámara USB	5 V	Sin medir	0,50 A (aprox)	Fuente 1 (vía USB)
Servo SG90 (reposo)	4,8–6 V [2]	0,01 A (aprox)	—	Fuente 2
Servo SG90 (bloqueado)	4,8–6 V [2]	—	0,65 A (aprox)	Fuente 2
Rama 1: Raspberry Pi y cámara	5 V	0,90 A (estimada)	2,5 A	Fuente 1: 5 V / 2,5 A
Rama 2: cuatro servos, un servo activo por vez	5 V	0,68 A (estimada)	0,65 + 3 × 0,01 = 0,68 A	Fuente 2
Rama 2: cuatro servos bloqueados a la vez (peor caso)	5 V	—	4 × 0,65 = 2,60 A	Fuente 2: ≥ 3 A (margen de 15 %)

Tabla 4. Tensión y corriente por componente y dimensionamiento de las fuentes

La Fuente 1 coincide con la capacidad de 2,5 A que el fabricante recomienda para la Raspberry Pi 3, que incluye hasta 1,2 A para periféricos USB [1]; el consumo estimado de la placa y la cámara (0,90 A) queda dentro de ese valor. La Fuente 2 se dimensiona para el peor caso, con los cuatro servos bloqueados a la vez (2,60 A) y un margen de 15 %, lo que lleva a 3 A y justifica el intervalo de 2 a 3 A del Plan. En operación normal se mueve una única compuerta por vez, por lo que el consumo esperado de la rama es de aproximadamente 0,68 A. Al usar una fuente de computadora ATX, además, deberá verificarse la capacidad del riel de +5 V en su etiqueta y conectar la señal PS_ON a tierra para que arranque fuera del gabinete.

4.4 Documentación del Código

El código del módulo se incorporó al repositorio del proyecto [16] en la carpeta raspberry/ Los fragmentos corresponden a esa versión, salvo el módulo de servos, aún no incorporado al

repositorio; se omiten partes accesorias. El panel de prueba fue reemplazado luego en el repositorio por el *dashboard* definitivo.

4.4.1 Código que controla el hardware

Cámara. El archivo `ecosort_pi.py` abre la cámara con la interfaz Video4Linux2 (V4L2) de Linux a través de OpenCV [7]. Se solicita el formato de compresión MJPG (*Motion JPEG*) para que la cámara comprima la imagen y reduzca la carga del bus USB y de la CPU, se fija la resolución en 640 × 480 (`cv2.VideoCapture(cam_id, cv2.CAP_V4L2)`) y se limita el almacenamiento intermedio a un cuadro para procesar siempre la imagen más reciente.

La lente M12 de la cámara tiene enfoque manual. Para ajustarlo, el archivo `vista_en_vivo.py` transmite el video por HTTP en el puerto 8000 y superpone un indicador de nitidez, calculado como la varianza del operador Laplaciano sobre la imagen en escala de grises: a mayor valor, mayor enfoque (`cv2.Laplacian(gris, cv2.CV_64F).var()`).

Detección de presencia y conteo. La clase `Fondo` memoriza la escena vacía durante los dos primeros segundos y mide, en cada cuadro, qué fracción de píxeles difiere de ella. Si el cambio supera el 3 % durante tres cuadros consecutivos, se considera que hay un objeto. Para contarlo se exige además que tres cuadros consecutivos arrojen la misma clase con confianza igual o superior al 60 %; una vez contado, no se vuelve a contar hasta que el objeto se retire (ocho cuadros sin cambio). La Figura 3 resume esta lógica, que se implementa con los umbrales `UMBRAL_CAMBIO = 0.03`, `FRAMES_PRESENTE = 3` y `FRAMES_AUSENTE = 8`.

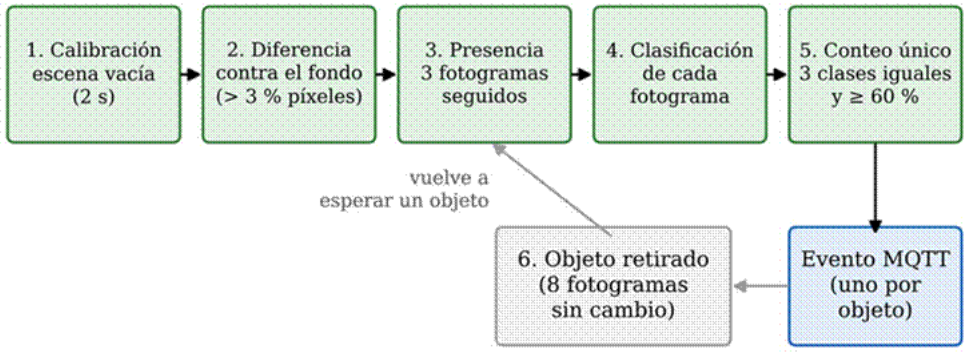


Figura 3. Lógica de detección de presencia y conteo único por objeto

Servos. El archivo `servos.py` define la clase `Compuertas`, que controla un servo SG90 por compuerta con la biblioteca gpiozero [8], usando el demonio pigpio [9] para generar la señal PWM por hardware y evitar el temblor (*jitter*) de la generación por software, tal como se indicó en el Plan. Las compuertas se abren de a una por vez mediante un cerrojo, para limitar el pico de corriente (sección 4.3), y se sueltan al cerrar para que el servo no consuma ni vibre en reposo. El módulo incluye un modo de prueba (`--test`) y otro de calibración de ángulos (`--angulo`).

4.4.2 Código que controla las comunicaciones

Broker. Mosquitto 2.x acepta por defecto únicamente conexiones locales. El archivo `raspberry/mosquitto/ecosort.conf`, que se copia en `/etc/mosquitto/conf.d/`, habilita el puerto 1883 en todas las interfaces (`listener 1883`) y las conexiones anónimas (`allow_anonymous true`), decisión que se acepta porque el tráfico no sale de la red que crea la Raspberry Pi (que a su vez debe estar protegida con contraseña, según el requerimiento 4.3 del Plan).

Publicador. La clase `EcoSortMQTT`, del archivo `ecosort_mqtt.py`, usa la biblioteca paho-mqtt [10]. Se conecta de forma asíncrona (`connect_async` y `loop_start`) para no bloquear la inferencia y reintentar si el *broker* aún no arrancó. Declara un mensaje de última voluntad (*Last Will and Testament*, LWT) con el que el *broker* publica `offline` si el proceso termina sin despedirse, y publica `online` retenido al conectarse. Los tópicos son los siguientes:

- `ecosort/<dispositivo>/eventos`: un mensaje por residuo contado, con calidad de servicio (QoS) 1.
- `ecosort/<dispositivo>/vivo`: lo que la cámara ve en ese instante, unas tres veces por segundo, con QoS 0, porque la pérdida de un mensaje es irrelevante.
- `ecosort/<dispositivo>/estado`: `online` u `offline`, retenido.

El identificador único del evento (`evento_id`) permite que el receptor descarte duplicados que el protocolo puede reenviar con QoS 1. La marca de tiempo del dispositivo se registra junto con la de recepción porque la Raspberry Pi 3 no tiene reloj de tiempo real y su hora puede ser incorrecta al arrancar sin red.

4.4.3 Código del Sistema y la interfaz web

El archivo `dashboard.py` es un servidor web de un único archivo, escrito solo con la biblioteca estándar de Python, SQLite [11] y paho-mqtt. Recibe los eventos por MQTT, los guarda en la base `eventos.db` y sirve la página del panel, que contiene el HTML, las hojas de estilo (CSS) y el código JavaScript embebidos. La Tabla 5 enumera sus rutas.

Ruta	Función
`GET /`	Entrega la página del panel.
`GET /api/resumen`	Devuelve, en notación de objetos de JavaScript (JSON), el total, el conteo por clase y los últimos registros.
`GET /api/stream`	Canal de eventos enviados por el servidor (Server-Sent Events, SSE) con el estado en vivo y las altas de registros.
`GET /api/descargar.csv`	Descarga todos los eventos en formato de valores separados por comas (CSV) apto para planillas de cálculo en español.
`POST /api/reiniciar`	Borra los datos registrados; la página solicita confirmación previa.

Tabla 5. Rutas HTTP del panel web de prueba

La inserción usa `INSERT OR IGNORE` sobre una columna `evento_id` única, de modo que un mensaje repetido no se registra dos veces. Cada alta se difunde a todos los navegadores conectados mediante SSE.

El navegador abre un único canal SSE y reacciona a tres tipos de mensaje: `vivo`, que muestra la clase y la barra de confianza; `evento`, que actualiza el conteo y la tabla; y `estado`, que cambia el indicador de conexión del detector. El diseño se adapta a pantallas de distinto tamaño y respeta el esquema de colores claro u oscuro. La descarga genera un CSV con separador punto y coma, coma decimal y marca de orden de bytes (BOM), para que Excel respete acentos y decimales en configuración regional española.

4.4.4 Software para PC o dispositivo móvil

No se desarrolló software específico para PC o dispositivo móvil: la interfaz es la página web de la sección 4.4.3, accesible desde el navegador de cualquier equipo conectado a la red de la Raspberry Pi en el puerto 8080. El código de interfaz es el HTML, CSS y JavaScript embebido en `dashboard.py`, y el de comunicación con la placa es la suscripción MQTT (`on_connect`, `on_message`) y el canal SSE.

4.5 Modelo de clasificación y optimización para la Raspberry Pi

Datos. Se utilizó TrashNet [4], con 2527 imágenes de objetos sobre fondo claro en seis clases (cartón, vidrio, metal, papel, plástico y basura). Las imágenes se dividieron de forma estratificada en 70 % para entrenamiento, 15 % para validación y 15 % para prueba, con semilla fija (42) para que la división sea reproducible.

Entrenamiento. Se empleó aprendizaje por transferencia sobre MobileNetV2 [3] preentrenada con ImageNet, en dos etapas: primero se entrenó solo la cabeza de clasificación (promedio global, *dropout* de 0,2 y capa densa de seis salidas) con la base congelada, y luego se ajustaron las últimas 40 capas de la base con una tasa de aprendizaje de $1 \cdot 10^{-5}$. Se aplicó aumento de datos (volteos, rotación, zoom, brillo y contraste), pesos por clase para compensar la menor cantidad de imágenes de basura y detención temprana. La normalización de los píxeles está incluida en el modelo, de modo que la Raspberry Pi le entrega la imagen en color de 0 a 255. El entrenamiento se ejecutó en Google Colab con el cuaderno `entrenar_ecosort.ipynb`.

Exportación. El modelo se convirtió a TensorFlow Lite [6] en dos variantes: números de punto flotante de 32 bits (fp32) y cuantización posterior al entrenamiento a enteros de 8 bits (int8), calibrada con 200 imágenes representativas del conjunto de entrenamiento. El tamaño de los archivos es de 8,9 MB para fp32 y de 2,7 MB para int8.

Resultados. En el conjunto de prueba el modelo alcanzó una exactitud aproximada del 79 %, sin pérdida atribuible a la cuantización. La clase con menor desempeño fue metal, con precisión de 0,675. La Tabla 6 y la Figura 4 comparan las dos variantes ejecutadas en la Raspberry Pi 3 con cuatro hilos, el delegado XNNPACK y 50 inferencias de medición tras un calentamiento. La variante int8 reduce la latencia media de 81 ms a 31 ms y permite unos 32 cuadros por segundo, de modo que la inferencia no limita el requerimiento de latencia de 2 a 4 segundos establecido en el Plan; la latencia de punta a punta se medirá en la próxima etapa.

Variante	Latencia media	Percentil 95	Cuadros por segundo	Memoria pico (RAM)
fp32	81 ms	87 ms	≈ 12	171 MB
int8 (elegida)	31 ms	40 ms	≈ 32	155 MB

Tabla 6. Latencia y memoria de las variantes del modelo en la Raspberry Pi 3

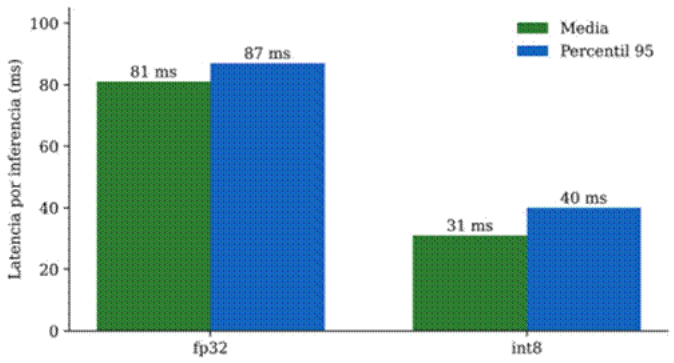


Figura 4. Latencia de inferencia por imagen en la Raspberry Pi 3 según la variante del modelo

Limitaciones. TrashNet contiene un único objeto sobre fondo blanco, condición distinta de la de la cámara instalada en el gabinete, por lo que se espera una caída de la confianza en condiciones reales. Esa diferencia se atenderá con un conjunto de datos propio, que incluirá la clase orgánico y fotografías tomadas con la cámara y el fondo definitivos. El mapeo de las clases de TrashNet a las categorías del producto (cartón y papel a papel, plástico, vidrio) deja sin compuerta a las clases metal y basura mientras no se defina su destino.

4.6 Pruebas realizadas

Se realizaron las siguientes pruebas funcionales en la Raspberry Pi:

- Comunicación MQTT: publicación y suscripción entre una PC con un cliente MQTT gráfico y la Raspberry Pi, con la placa alimentada por su fuente de red eléctrica y conectada por Wi-Fi.
- Captura: fotografías con la cámara USB tomadas desde la Raspberry Pi y transferidas a la PC, y transmisión de video en vivo por HTTP para ajustar el enfoque.
- Inferencia: banco de pruebas de latencia y memoria (Tabla 6) y clasificación en vivo de objetos cotidianos presentados a la cámara (frasco de vidrio, vaso de plástico, termo metálico, papel y cartón).
- Cadena completa: detección de presencia, clasificación, conteo, publicación MQTT y visualización en el panel, con conteo por clase, estado en vivo, descarga de CSV y reinicio de los datos, registrada en fotografías (Figura 6) y en video (sección 5.3).

No se midió de manera sistemática la precisión del sistema con la cámara instalada ni la tasa de eventos falsos; esas mediciones forman parte del Avance de Noviembre.

4.7 Problemas encontrados y soluciones

La Tabla 7 describe los problemas más importantes encontrados hasta el momento, su solución o la alternativa prevista.

Problema	Causa	Solución o alternativa
TrashNet no contiene la clase orgánico	El conjunto público solo incluye seis clases de materiales	Pendiente: construir un conjunto propio con fotografías de la cámara, incluida la clase orgánico.
Menor confianza con la cámara real	Las imágenes de entrenamiento tienen un solo objeto sobre fondo blanco	Umbral de confianza del 60 % y exigencia de tres cuadros consecutivos; el reentrenamiento con datos propios es la mejora principal prevista.
Un mismo objeto generaba varios conteos	La clasificación se ejecuta en cada cuadro	Detección de presencia por diferencia de fondo y estado «contado» hasta que el objeto se retire.
El adaptador se cerraba (exit 1) si el broker no estaba disponible al arrancar	paho-mqtt no reintenta la primera conexión por defecto	retry_first_connection=True; se agregó una verificación a la prueba de extremo a extremo para que esto no pueda volver a pasar sin que la prueba falle.
La cámara no se abría (`/dev/video0` ocupado)	Otro proceso, como la vista en vivo, mantenía el dispositivo	Un único proceso por vez accede a la cámara; se cierra el anterior antes de iniciar el detector.
Hora incorrecta en los eventos	La Raspberry Pi 3 no tiene reloj de tiempo real	Se registra la hora del dispositivo y la de recepción por separado.
El esquema de datos y el mapeo de clases podían divergir entre el entrenamiento del modelo y el contrato	El mapeo vivía en dos lugares distintos (vision/classes.py y schema/clases.json) sin nada que los mantuviera iguales	schema/clases.json pasó a ser la fuente única; un test de contrato falla si vision/classes.py llega a divergir.

Tabla 7. Problemas encontrados y soluciones

4.8 Planificación de las partes faltantes

La Tabla 8 presenta las tareas pendientes del módulo hasta el Avance de Noviembre, con el consumo asociado a cada hardware.

Período	Tarea	Hardware y consumo asociado
Octubre	Verificar la capacidad del riel de +5 V de la fuente de computadora y habilitar su encendido (PS_ON a tierra)	Fuente 2: riel de +5 V ≥ 3 A

Octubre	Correr el playbook de Ansible contra una Raspberry Pi real y registrar el resultado en la bitácora — hasta ahora solo se verificó su sintaxis y sus plantillas.	—
Octubre	Probar los servos con el módulo `servos.py`, calibrar ángulos y medir la corriente real de reposo y de bloqueo	4 servos SG90: 0,68 A esperados con un servo activo; 2,60 A en el peor caso
Octubre	Montar el circuito con la alimentación separada y la tierra común, y comprobar que la Raspberry Pi no se reinicie al accionar los servos	Fuente 1: 5 V / 2,5 A; Fuente 2: 5 V / 3 A
Octubre	Tomar fotografías propias con la cámara instalada (50 a 100 por clase, incluyendo orgánico) y de situaciones sin residuo	Cámara USB: hasta 0,50 A
Fines de octubre	Reentrenar el modelo con el conjunto propio y recalcular exactitud y matriz de confusión	—
Noviembre	Calibrar los umbrales de presencia y de confianza con la cámara real, medir los eventos falsos por sesión y la latencia de punta a punta hasta la señal al servo	Raspberry Pi y cámara: 0,90 A estimados
Noviembre	Automatizar el arranque del módulo como servicio del sistema y actualizar la guía de instalación	—
Noviembre	Evaluar sumar Prometheus y node_exporter para la salud del dispositivo (ADR 0004/0005), fuera del alcance de esta entrega.	—

Tabla 8. Planificación de las tareas pendientes del módulo de la Raspberry Pi

5. Documentación Relacionada

5.1 Fotografías del hardware

La Figura 5 muestra el montaje de prueba de la Raspberry Pi 3, alimentada por su fuente a través del conector micro-USB y con la cámara USB conectada.

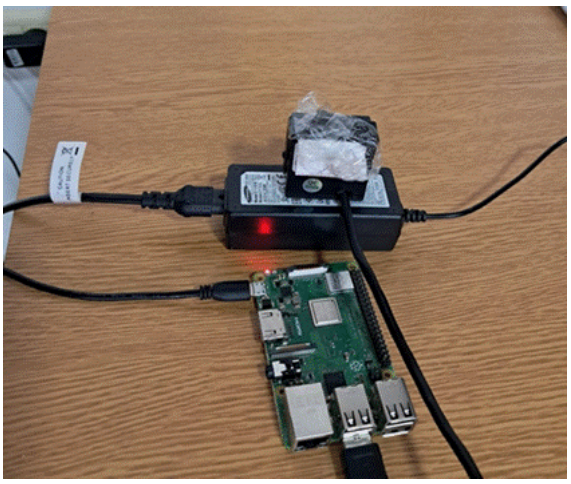


Figura 5. Montaje de prueba: Raspberry Pi 3 con su fuente de alimentación y la cámara USB conectada

5.2 Capturas de la interfaz web

La Figura 6 muestra el panel web de prueba (sección 4.4.3) con el detector conectado, fotografiado desde la pantalla de la notebook. Se observa el estado en vivo con la clase detectada («Vidrio», confianza de 71 %), el total de cinco residuos con el conteo por clase y la tabla de últimos registros. Los botones superiores permiten descargar los datos en CSV y reiniciarlos.

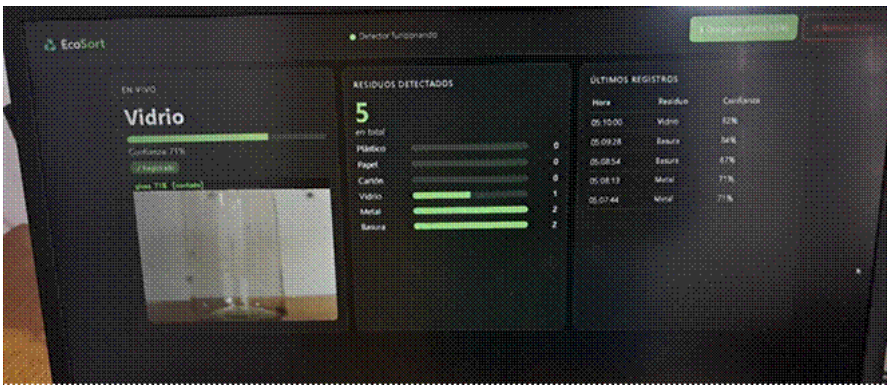


Figura 6. Panel web de prueba en funcionamiento, con el detector conectado

5.3 Video de las pruebas

Video de las pruebas del sistema, de aproximadamente dos minutos, en el que se presentan a la cámara objetos cotidianos (un frasco de vidrio, un vaso de plástico, un termo metálico, papel y cartón) y se observa cómo el panel actualiza la clase detectada y el conteo: https://drive.google.com/file/d/12eBN0zDp_0o9ujvR2kKs-BwtIZuZW06t/view?usp=drive_link

Video de presentación del proyecto, entregado con el Plan de Proyecto: https://www.youtube.com/watch?v=tk-f81f-kwM

5.4 Bitácora y repositorio

- Bitácora del proyecto: https://github.com/tpII/2026-g5-ecosort/blob/main/BITACORA.md.
- Repositorio de código fuente: https://github.com/tpII/2026-g5-ecosort.
- Guía de instalación de la Raspberry Pi: https://github.com/tpII/2026-g5-ecosort/blob/main/docs/guia-instalacion-raspberry.md.

La cátedra cuenta con acceso de edición a la bitácora y al repositorio.

6. Bibliografía

[1] Raspberry Pi Ltd. «Power supplies». Raspberry Pi documentation. https://github.com/raspberrypi/documentation/blob/master/documentation/asciidoc/computers/raspberry-pi/power-supplies.adoc.

[2] Waveshare. «SG90 Servo». https://www.waveshare.com/SG90-Servo.htm.

[3] Sandler, M.; Howard, A.; Zhu, M.; Zhmoginov, A.; Chen, L.-C. «MobileNetV2: Inverted Residuals and Linear Bottlenecks». arXiv:1801.04381, 2018. https://arxiv.org/abs/1801.04381.

[4] Thung, G.; Yang, M. «Classification of Trash for Recyclability Status». Stanford CS229, 2016. Conjunto de datos TrashNet: https://github.com/garythung/trashnet.

[5] Eclipse Foundation. «Eclipse Mosquitto: an open source MQTT broker». https://mosquitto.org/

[6] Google. «LiteRT (anteriormente TensorFlow Lite)». https://developers.google.com/edge/litert.

[7] OpenCV. «Open Source Computer Vision Library». https://opencv.org/

[8] Jones, B.; Thomas, D. et al. «gpiozero: API – Output Devices (AngularServo)». https://gpiozero.readthedocs.io/en/stable/api_output.html.

[9] Joan N. N. «pigpio library». http://abyz.me.uk/rpi/pigpio/

[10] Eclipse Foundation. «Eclipse Paho MQTT Python client». https://github.com/eclipse-paho/paho.mqtt.python

[11] SQLite Consortium. «SQLite». https://www.sqlite.org/

[12] JSON Schema. «JSON Schema: A Media Type for Describing JSON Documents», Draft 2020-12. https://json-schema.org/

[13] Docker Inc. «Compose file reference». https://docs.docker.com/reference/compose-file/

[14] freedesktop.org. «systemd.service — Service unit configuration». https://www.freedesktop.org/software/systemd/man/systemd.service.html

[15] Red Hat. «Ansible Documentation». https://docs.ansible.com/

[16] Grupo G5. «EcoSort: repositorio del proyecto». https://github.com/tpII/2026-g5-ecosort.